FinGPT Two-Agent Signal Pipeline:
Calibrated Trading Signals from Financial News via Real Token-Level Log-Probabilities and PMI Correction

May 2026

Abstract

We present a local, inference-only pipeline that converts financial news articles into calibrated directional trading signals using a fine-tuned 8-billion-parameter reasoning model (DeepSeek-R1-Distill-Llama-8B). Two specialised agents run sequentially on a shared vLLM engine: Agent 1 extracts structured sentiment from the article text; Agent 2 reasons about the appropriate one-week trading action. Neither agent outputs numbers or JSON directly. Instead, the model's genuine token-level log-probabilities are read from the vLLM `prompt_logprobs` interface and converted to calibrated probabilities via temperature-scaled softmax. A key engineering challenge — the dominance of an unconditional language-model prior that caused Agent 2 to return HOLD for every article — was resolved by Pointwise Mutual Information (PMI) normalisation. A backtest over 291 signals across all 30 Dow Jones constituents (February–April 2024) yields a win rate of 53.6% on active trades, an annualised Sharpe ratio of 0.67, and a maximum drawdown of −31.4%.

---

1. INTRODUCTION

Translating unstructured financial news into actionable trading signals is a long-standing problem in quantitative finance. Large language models (LLMs) offer a natural fit: they encode broad financial knowledge and can reason over narrative text. However, deploying them reliably for signal generation raises two practical difficulties. First, asking a model to self-report its confidence numerically — as a JSON field or a plain number — produces values that correlate poorly with its actual token probabilities, because the model is performing next-token prediction rather than genuine introspection. Second, unconditional language-model priors at syntactically structured decision points can overwhelm the news-driven signal, causing the model to output the same class regardless of article content.

This paper describes a pipeline that addresses both difficulties. All decision logic is implemented in deterministic Python; the model is used only as a probability estimator whose beliefs are read directly from vLLM's `prompt_logprobs` interface. A two-phase chain-of-thought approach separates reasoning from scoring. PMI normalisation removes unconditional token priors so that only news-driven information determines the final signal.

2. SYSTEM OVERVIEW

The pipeline comprises two sequential agents sharing a single vLLM engine instance. Agent 1 (the *extractor*) receives a raw news article and produces a `NewsFingerprint`: a structured record containing the article headline, the companies mentioned, extracted event keywords, and a calibrated three-class sentiment distribution over POSITIVE, NEGATIVE, and NEUTRAL. Agent 2 (the *reasoner*) receives the `NewsFingerprint` and produces a `TradingSignal`: a directional action (long, neutral, or short) with a calibrated confidence score.

Each article requires five sequential vLLM engine calls: Call 1 performs guided JSON decoding for fact extraction. Calls 2 and 3 form Agent 1's two-phase sentiment scoring loop. Calls 4 and 5 form Agent 2's two-phase strategy scoring loop. Articles are processed in batches of ten; each batch therefore makes five `engine.generate()` calls rather than fifty, saturating GPU throughput without exhausting KV-cache memory.

3. TECHNOLOGY STACK

Table 1 summarises the libraries and services used. The foundation model is DeepSeek-R1-Distill-Llama-8B, a chain-of-thought reasoning model based on the Llama architecture, fine-tuned for financial text. The recommended execution environment is Google Colab with an A100 or T4 GPU.

Table 1: Technology stack by layer.

Layer	Technology and role
LLM runtime	vLLM — local high-throughput serving; `prompt_logprobs`, guided decoding
Foundation model	DeepSeek-R1-Distill-Llama-8B (fine-tuned)
Schema	Pydantic v2 — `NewsFingerprint`, `TradingSignal`
Tokenisation	HuggingFace Transformers — BPE token ID resolution
Dataset	HuggingFace Datasets — FinGPT Dow 30 corpus
Price data	yfinance — daily OHLC, close-to-close returns
Data wrangling	pandas + NumPy
Config	python-dotenv

4. TWO-PHASE LOG-PROBABILITY SCORING

4.1 Motivation

The central design principle is that the model's genuine beliefs should be read from its computation graph, not solicited as text. When a local LLM is asked to output `{"confidence": 0.87}`, the number it produces reflects conversational conventions about how confident-sounding responses are phrased, not the actual token probabilities computed in the forward pass. The `prompt_logprobs` interface in vLLM provides access to the real values.

4.2 Phase 1: Chain-of-Thought Generation

Both agents use an identical first phase. The input is fed with a prompt that instructs the model to reason inside `<think>...</think>` tags. vLLM is configured to stop generation the moment it emits the closing tag:

```
SamplingParams(
    max_tokens=1024,
    temperature=0.0,
    stop=["</think>"]
)
```

The reasoning trace is preserved as an audit trail but plays no direct role in the scoring step that follows.

4.3 Phase 2: Scoring via `prompt_logprobs`

For each candidate class c (e.g. POSITIVE, NEGATIVE, NEUTRAL for Agent 1), a scoring prompt is constructed by appending the generated chain-of-thought, a decision prefix, and the candidate label to the original prompt. All candidate prompts are submitted in a single batched `engine.generate()` call with:

```
SamplingParams(
    prompt_logprobs=1,
    max_tokens=1,
    temperature=0.0
)
```

vLLM includes the log-probability of the actual prompt token at every position, so the pipeline reads $\log P(c \mid \text{context})$ directly from the model's computation graph. For multi-token classes such as POSITIVE (tokenising as `["␣POS", "ITIVE"]`), per-token log-probabilities are summed:

$$\log P(\text{POSITIVE} \mid \text{ctx}) = \sum_{t \in \text{tokens(POSITIVE)}} \log P(t \mid \text{ctx}_{<t})$$

4.4 Calibration

The resulting log-probability vector $\boldsymbol{\ell}$ is converted to a probability distribution via temperature-scaled softmax:

$$p_i = \frac{\exp(\ell_i / T)}{\sum_j \exp(\ell_j / T)}, \qquad T = 1.2$$

A temperature $T > 1$ flattens the distribution, reducing overconfidence on ambiguous inputs. The argmax label and its probability become the `label` and `confidence` fields of the output schema.

5. PMI PRIOR CORRECTION

5.1 The HOLD-Dominance Problem

Agent 2 must choose among three actions: BUY, HOLD, and SELL. An early design used the full words as scoring tokens. HOLD dominates at the decision context `"Strategy: "` because its unconditional language-model prior is far higher than BUY or SELL at this syntactic position in financial text. Switching to single ASCII letters A (BUY), B (HOLD), C (SELL) reduced but did not eliminate the problem. At the decision prefix `"The answer is ("`, the token `"B"` retains an unconditional prior of approximately 0.75. This is not news-driven: the model has encountered `"(B)"` as a multiple-choice answer format throughout its training corpus. The prior completely swamps the news signal, causing even strongly bullish articles to receive a HOLD signal.

5.2 PMI Normalisation

Pointwise Mutual Information removes the unconditional prior by measuring only the news-driven component of each token's log-probability:

$$\mathrm{PMI}(c,\, a) = \log P(c \mid a) - \log P(c \mid \emptyset)$$

where a denotes the article and $\emptyset$ denotes a neutral null article (*"No specific news available."*). The null-context log-probabilities $\log P(c \mid \emptyset)$ are computed once per engine session and cached. Per-article PMI logits are then formed by simple subtraction:

```
null_lp   = _compute_null_logprobs()
pmi       = [raw[i] - null_lp[i] for i in range(3)]
probs     = softmax(pmi, T=CALIBRATION_T)
direction = STRATEGY_SET[argmax(probs)]
```

After this correction, a neutral article produces near-zero PMI logits for all three tokens and correctly receives HOLD. A strongly bullish article produces a clearly positive PMI logit for A, yielding BUY regardless of the B-prior.

5.3 Empirical PMI Variant

When no GPU is available for model-computed null logprobs, an empirical approximation estimates $\log P(c \mid \emptyset)$ as the column mean of the `signal_logits` across all rows in an existing result CSV. This is the Track A path in `resume_backtest.ipynb`. The approximation is less precise than the model-computed null but sufficient for post-hoc recalibration.

6. BACKTEST

6.1 Experimental Setup

The backtest uses the HuggingFace dataset `FinGPT/fingpt-forecaster-dow30-202305-202405`, which provides structured multi-headline blocks for each Dow 30 constituent on a weekly cadence. Realised returns are fetched from Yahoo Finance using `interval="1d"` over a seven-day window, producing up to five trading-day bars; the close-to-close return across those bars is the realisation metric. A weekly interval over a seven-day window returns at most one bar and cannot produce a return; the daily interval was a deliberate fix to this failure mode.

The evaluation window spans 291 signals across all 30 Dow Jones constituents from February to April 2024.

6.2 Results

Table 2 reports headline metrics. Long signals averaged +0.50% per trade; short signals averaged −0.12%. The model's long-side calls are credible, but short-side calls underperform and would benefit from a higher confidence threshold. Notably, the highest signal-confidence quartile produced

0.00% average return, which may indicate overconfidence at the top of the distribution and warrants further calibration study.

Table 2: Backtest results (Feb–Apr 2024, Dow 30).

Metric	Value
Total signals	291
Skipped	9
Active trades	179
Win rate (active)	53.6%
Mean return per trade	+0.22%
Annualised Sharpe	0.67
Max drawdown	−31.4%

6.3 Skip Reasons

Three failure modes cause a row to be skipped. `fingerprint_failed` indicates a parse error in Agent 1's guided JSON decoding call. `signal_failed` indicates that Agent 2 returned `None` due to a BPE logprob indexing failure. `price_fetch_failed` indicates that yfinance returned no usable data for the ticker and date window.

7. KEY DESIGN DECISIONS

Reading `prompt_logprobs` from vLLM gives the model's actual beliefs as computed by its forward pass, bypassing the noisy verbal generation channel through which self-reported confidence numbers pass.

The model reasons freely in Phase 1 and is scored in Phase 2 via a single deterministic lookup. This preserves interpretability through the CoT audit trail while keeping the decision mechanism noise-free.

Language model next-token probabilities at fixed syntactic positions are confounded by unconditional token priors. PMI removes that prior so that only the news-driven component determines the signal.

Every agent returns `None` on any parsing or validation failure rather than manufacturing a fallback. Skip reasons are recorded in the output CSV so pipeline health can be monitored systematically.

8. CONFIGURATION

Table 3 lists the primary environment variables that govern pipeline behaviour.

Table 3: Configuration variables.

Variable	Description
`FINGPT_MODEL_PATH`	Path to HuggingFace model weights (required)
`SHARE_SINGLE_LLM`	Share engine between agents to halve VRAM
`VLLM_ENABLE_V1_MULTIPROCESSING`	Set `0` in Colab to avoid `fileno` crash
`FINGPT_CALIBRATION_T`	Softmax temperature (default 1.2)
`FINGPT_LOGITS_MAX_TOKENS`	CoT token budget per article (default 1024)
`FINGPT_YF_CACHE_PATH`	Price cache path
`NEWS_PROVIDER`	`finnhub` or `alpaca` (live only)

9. CONCLUSION

We have described a two-agent pipeline that produces calibrated trading signals from financial news by reading genuine token-level log-probabilities from a local vLLM engine rather than soliciting

self-reported confidence values. The central engineering contribution is PMI normalisation, which resolves a systematic HOLD-dominance bias arising from the unconditional language-model prior at the scoring decision point. The backtest results over 291 Dow 30 signals are positive but modest: a win rate slightly above chance, an annualised Sharpe of 0.67, and meaningful per-ticker variation that suggests ticker-specific calibration could improve performance. Short-signal quality and the counterintuitive underperformance of the highest-confidence quartile are the primary areas for future work.